

PROJECT OF DEEP LEARNING

Abstract

This project investigates the use of deep learning to classify animal species into their respective families using a highly imbalanced image dataset containing 11,983 samples across 202 classes. The workflow included exploratory data analysis, outlier removal using the CLIP model, data augmentation, and model development. We evaluated five models, including both custom CNN architectures and pre-trained networks such as ResNet50 and VGG16. Among these, ResNet50 achieved the best performance, with a macro F1-score of 45.78% and accuracy of 48.98%. The results demonstrate the effectiveness of pre-trained models and robust preprocessing in handling complex multi-class classification tasks.

David Cascão, 20240851
Jan-Louis Schneider, 20240506
Emir Kamiloglu, 20240945
Marta Boavida, 20240519
Sofia Gomes, 20240848

Spring Semester 2024-2025

Table of Contents

- 1. Introduction 2
- 2. Explore Data Analysis 2
- 3. Pre-Processing 3
- 4. Models 4
- 5. Final model & Evaluation 6
- 6. Conclusion 6
- 7. References 7
- 8. Annexes 8

1 Introduction

In this project we developed a deep-learning model to predict the family of given species based on its image. To create this model, our group went through several phases, namely: data exploration, preprocessing, model development and finally, we analyzed the results and made visualizations to make it more intuitive. We used five models and in the end we compared the models in order to obtain the best results.

Through this report, you can get a detailed explanation of everything that was done to achieve our goal. The code that made this project possible is available at the following link https://github.com/Gomsofi06/DL_project.

2 Explore Data Analysis

To gain a foundational understanding of our dataset, we conducted an Exploratory Data Analysis on the metadata file. The dataset comprises 11,983 images and 7 features, including 2 numerical and 5 categorical attributes.

As an initial step, we analyzed the phylum distribution and identified five distinct categories: Chordata, Arthropoda, Cnidaria, Mollusca, and Echinodermata [1]. The majority of images (9,952) belong to the Chordata phylum, while Echinodermata is the least represented, with only 60 samples. We further examined the distribution of families within each phylum and found a total of 202 unique families. Chordata not only dominates in sample count but also in diversity, with 166 unique families. In contrast, Echinodermata contains only 1 family, highlighting a significant class imbalance in both quantity and variety. Next, we assessed data quality by inspecting for missing values and duplicates and confirmed that the dataset was already clean and well-structured.

To identify potential outliers, we first visualized all images from the Salmonidae family as an example [2] using a grid layout. This approach helped us spot mislabeled samples, scientific illustrations instead of real animal images, low-quality or noisy images, and non-animal content such as background artifacts or text. Based on this initial inspection, we conducted a more thorough manual review of each family folder to better understand the extent and nature of outliers across the dataset.

Lastly, we evaluated the distribution of samples per family [3] and confirmed that the dataset is highly imbalanced. A small number of families are heavily represented (with up to 300 images), whereas most families have fewer than 50 samples. This long-tail distribution presents challenges for model training, which we addressed in the subsequent preprocessing phase. To prepare the data for model development, we concluded this exploratory section by splitting the dataset into training, validation, and test sets, allocating 75% for training, 15% for validation and 10% for testing. Given the class imbalance, we employed a stratified split strategy to maintain the original class distribution across all subsets, ensuring that no family is over or under represented in any split. Finally, we used `random_state=42` to ensure reproducibility as well as that model comparison is done using the same split.

3 Pre-Processing

Our main goal in preprocessing was to effectively identify and remove outliers from the dataset. Having limited experience with image preprocessing techniques, our initial approach was rather primitive - we tried to detect anomalous images by analyzing pixel-level features such as RGB values, HSV color space distributions and other statistical properties. We assumed that images would be outliers if they showed extreme values in various feature dimensions. After viewing several extreme cases, we quickly realized that this approach was inadequate for capturing the nuanced differences between valid and anomalous animal photographs in our dataset.

To support our image pre-processing workflow, we used pre-trained models in visual data tasks. This helped us design an effective workflow, based on proven methods and taking advantage of existing tools, rather than starting from scratch. In addition, we created a pre-processed data framework with metadata to store all the transformations from our pre-processing, allowing us to revert changes if necessary and providing structured data for modeling.

During our exploration phase, we tested several experimental approaches that ultimately weren't incorporated into our final pipeline. One such attempt was background removal using a pre-trained U²-Net model [6] via the rembg library to segment and isolate animals from their backgrounds. We applied this process only to images previously classified as 'animal' by our filtering algorithm. Images meeting all quality requirements were labeled as 'good', while those failing quality checks were marked as 'bad' and excluded [5]. While we achieved excellent visual results with this background removal approach, these enhanced images didn't improve model performance as expected. We suspect this occurred because the background-removed images were only present in the training split, making it difficult for the model to reconcile these processed images with unprocessed validation images containing backgrounds. This experiment can be found in the preprocessing_testing notebook.

We also explored the use of GANs [3] as a potential method for generating new samples to address class imbalance. However, despite extensive training over many epochs, the results were not satisfactory, since the generated images lacked visual quality and failed to capture meaningful features of the animal species [8]. As a result, we decided not to include GAN-based augmentation in our final pipeline. This experiment can be found in the preprocessing_testing notebook.

In our final pipeline, we implemented the pre-trained CLIP model from OpenAI [2] to automatically identify and filter outliers. As highlighted in the EDA section, the dataset contained a significant number of problematic images including non-animal content such as text documents or landscape scenes. CLIP's ability to associate images with textual descriptions proved invaluable for this task.

Using CLIP, we performed a binary classification task distinguishing between animal and non-animal images. We defined two sets of prompts to guide the model's predictions: the animal prompts described realistic photos of animals, while the non-animal prompts covered content such as scientific illustrations, geographical maps, or landscapes with no animals. Each image in the dataset was processed through CLIP's image encoder and compared to the pre-defined prompts. We configured the

model to return the top-3 matching text prompts along with their confidence scores, and based on the highest-ranking match and a pre-defined confidence threshold of 0.5, images were classified as either 'animal' or 'non-animal' [4]. Following classification, we grouped the results into grids for visual inspection. Each image was annotated with its top predicted label and confidence score, and saved into separate folders for animal and non-animal categories. This helped us review the images to find the optimal threshold value. Finally, we updated the preprocessing metadata DataFrame to include the CLIP classification results, top-3 prediction scores, and the corresponding classification across all 3 splits.

To address the severe class imbalance identified during EDA, we implemented a data augmentation pipeline using the AugmentationProcessor to synthetically increase the representation of underrepresented families. We began by calculating an augmentation plan to determine how many additional images were needed per class to achieve a more balanced dataset. For each image, we applied a series of randomized transformations using the Albumentations library [8] to generate realistic variations of existing samples within the same class. The applied transformations included resizing, affine transformations with scaling and rotation, horizontal flips, brightness and contrast adjustments, Gaussian blur, and elastic transformations [6]. The newly generated images were saved, and the metadata DataFrame was updated with the corresponding file paths and transformation parameters. This approach allowed us to balance the dataset more effectively, reduce overfitting, and improve model generalization by introducing controlled variability.

As a final preprocessing step, we standardized all images to a uniform size of 224×224 pixels, which is the required input dimension for many pre-trained convolutional neural networks, including those used in our modeling phase. This resizing was consistently applied across all datasets - training, validation, and test - to ensure uniformity and compatibility with our chosen model architectures. [9]

4 Models

In this section, we will describe our modeling approach. We implemented three different model approaches. First, we built a baseline model to establish a general benchmark. Then, we refined this model using advanced parameter tuning, resulting in two improved versions: v1 and v2. Finally, we applied two pre-trained models to compare results and further enhance performance.

The first step was further preparing the data to be compatible with the models. After some additional preprocessing — such as standardizing path formats and distinguishing between paths of original images and their augmented versions — we removed all images previously labeled as non_animal from every dataset. This focused approach enabled the models to allocate computational resources on learning the specific morphological characteristics that distinguish one animal species from another, rather than spending capacity on the broader animal versus non-animal classification. Finally, we applied label encoding to convert the species names into numerical target class values.

For the baseline and advanced models, we initially attempted to use the Keras library [1], to apply the concepts from the practical classes. However, we encountered unusual behavior in which the model consistently predicted only a single class. Despite multiple efforts to solve this issue, the only observed difference was the specific class being predicted to change to another one, not the behavior itself. Consequently, we switched to the PyTorch library [4], which provides a more flexible and advanced framework for CNN modeling. While PyTorch requires more manual configuration to be used, it allows for greater control over model architecture and training, whereas Keras is more user-friendly but offers less flexibility. We employed Convolutional Neural Networks (CNNs) for both the baseline and advanced models, with differences in parameters and architectural complexity. For the baseline model we used two convolutional layers followed by a fully connected output layer. The highest validation accuracy achieved with this model was 14.7%. After not improving the validation loss for three epochs, the model stopped using early stopping. The macro F1-score obtained was approximately 9%. As expected, the baseline model performed poorly and exhibited significant overfitting, but it provided a foundation upon which we could build more complex and tuned models.

For the advanced model v1, we added a third convolutional layer and applied dropout in an attempt to reduce overfitting. This model achieved a maximum validation accuracy of approximately 24%. However, after reaching a minimum validation loss of around 4, the loss began to increase, peaking at about 7.5. At the same time, training accuracy rose to 95%, clearly indicating overfitting. Nonetheless, the performance, especially in terms of validation accuracy, was clearly better than that of the baseline model.

For the advanced model v2, we applied more a sophisticated model architecture and hyperparameter tuning to improve performance. We increased the network depth to four convolutional layers and included two fully connected output layers. Additionally, we reduced the dropout rate and implemented a dynamic learning rate, which automatically decreased the learning rate after three epochs without improvement. This model achieved a better validation accuracy of 28.6% after 23 epochs, with validation loss stabilizing around 3.5. The final training accuracy was approximately 62%, indicating significantly less overfitting compared to previous models, while also demonstrating better overall validation performance.

Although a validation accuracy of 25–30% may seem low at first glance, it is important to consider that the model has to classify images into 202 distinct classes. Therefore, such results are within a reasonable range for models at this level of complexity. Furthermore, we evaluated not only top-1 predictions but also top-3 predictions. Since the model outputs a probability for each class, we examined the proportion of cases in which the correct class appeared among the top three predictions with the highest probabilities. This top-3 accuracy was 40%. So, while the model's top prediction was correct in 28% of cases, the correct class appeared in the top three predictions 40% of the time. This suggests that the model is often on the right track, and with further tuning, these results could potentially be converted into accurate predictions in the next model. Lastly, the model V2 achieved a macro F1-score of 21.29

As previously mentioned, we also tested two pre-trained models in order to

achieve better results. The first was ResNet50 [7], implemented using the PyTorch torchvision library, and trained with a dynamic learning rate. This model achieved a validation accuracy of approximately 56% and a minimum validation loss of 2.28, representing a significant improvement over the previous models. Considering the task involves over 200 classes, these results are quite acceptable. However, the model still exhibited overfitting, with a training accuracy of around 95%. The macro F1-score reached 45.78%, and the top-3 accuracy was 68%, indicating that in nearly 7 out of 10 cases, the correct class was among the top three predictions with the highest probabilities.

As a second pre-trained model, we tested VGG16, also from the PyTorch torchvision library [5], using the same training parameters. However, the results were underwhelming. This model achieved a maximum validation accuracy of only 21%, a validation loss of around 4, and a macro F1-score of just 11% — performing worse than some of our advanced custom CNN models, which was unexpected.

5 Final model & Evaluation

As shown in the graphs [10] and [11], our best-performing models based on the evaluation metrics were the advanced CNN version v2 and the pre-trained ResNet50 model. Among them, ResNet50 demonstrated the strongest overall performance, achieving a macro F1-score of 45% and an accuracy of 49%. Consequently, we selected this model as our final choice. Upon analyzing the distribution of predicted classes, we observed that no single class was predicted in more than 3% of the images. This indicates that the model has no significant bias toward any particular class.

Overall, our two best models — the pre-trained ResNet50 and the custom CNN v2 — both produced relatively balanced predictions, making them reasonable candidates for classifying animal families in this dataset.

6 Conclusion

In this project, we addressed the challenge of classifying animal families from a highly imbalanced dataset containing over 200 classes. We began with data exploration and applied a series of preprocessing steps, including the use of CLIP for automated outlier detection and data augmentation strategies to tackle class imbalance.

We experimented with both custom CNN models and pre-trained architectures. The custom CNNs showed gradual improvements across versions, with the v2 model significantly reducing overfitting and achieving better validation accuracy and macro F1-score compared to the baseline. These models helped us understand the impact of architectural tuning. Ultimately, the pre-trained ResNet50 model outperformed all others, achieving a macro F1-score of 45% and accuracy of 48.98% and we select it as our final model. While these results are promising, they also highlight opportunities for further improvement, particularly in enhancing model generalization and addressing class imbalance, which could be explored in future work.

References

- [1] François Chollet et al. *Keras*. 2015. URL: <https://keras.io> (visited on 04/01/2025).
- [2] *CLIP model*. URL: <https://openai.com/index/clip/>.
- [3] Ian Goodfellow et al. “Generative adversarial nets”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014, pp. 2672–2680. URL: <https://papers.nips.cc/paper/5423-generative-adversarial-nets>.
- [4] *Pytorch*. URL: <https://pytorch.org/>.
- [5] *Pytorch Model*. URL: <https://pytorch.org/vision/stable/models/generated/torchvision.models.vgg16.html>.
- [6] Xuebin Qin et al. “U²-Net: Going deeper with nested U-structure for salient object detection”. In: *Pattern Recognition* 106 (2020), p. 107404. DOI: [10.1016/j.patcog.2020.107404](https://doi.org/10.1016/j.patcog.2020.107404).
- [7] *ResNet*. URL: <https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html>.
- [8] Albumentations Team. *Albumentations Documentation*. Accessed: 2025-04-05. 2025. URL: <https://albumentations.ai/docs/>.

A Annexes

Figure 1: Family Tree

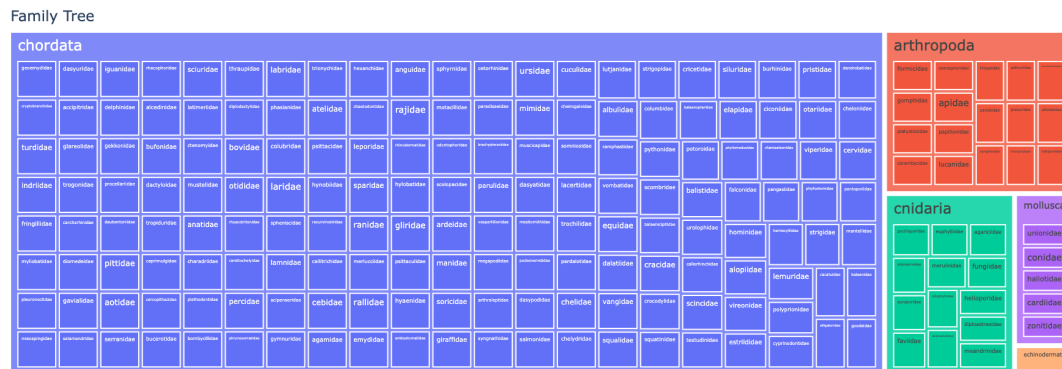


Figure 2: Salmonidae family

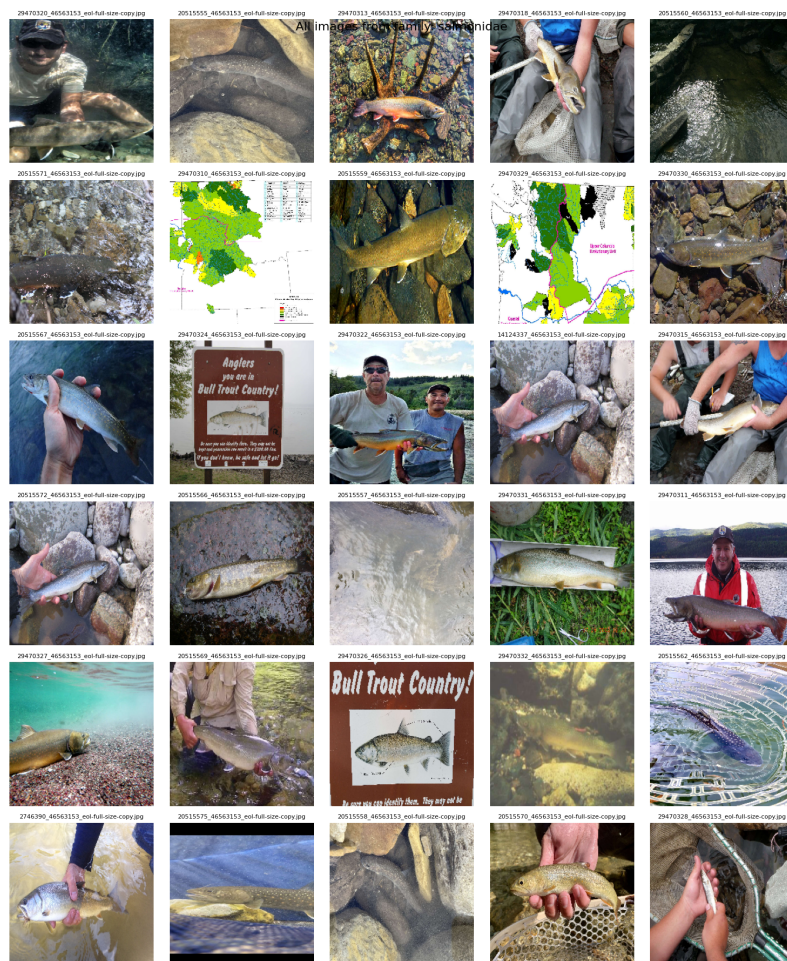


Figure 3: Distribution of Species

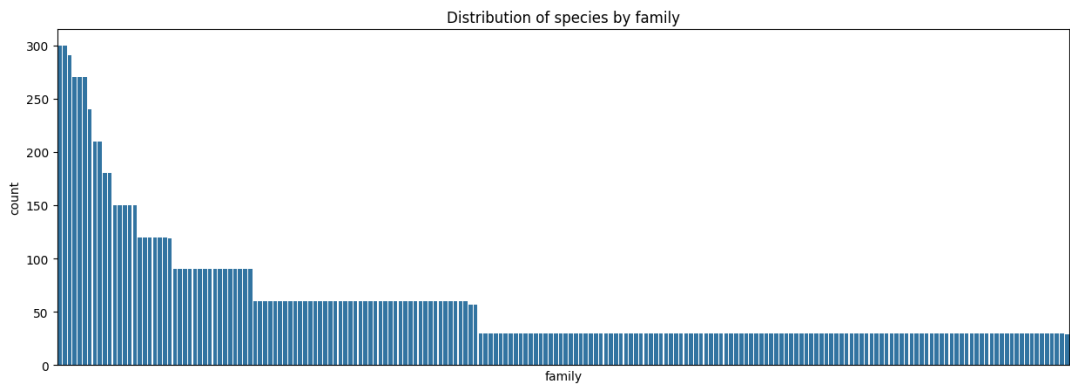


Figure 4: CLIP Results



Figure 5: Background Removal Results

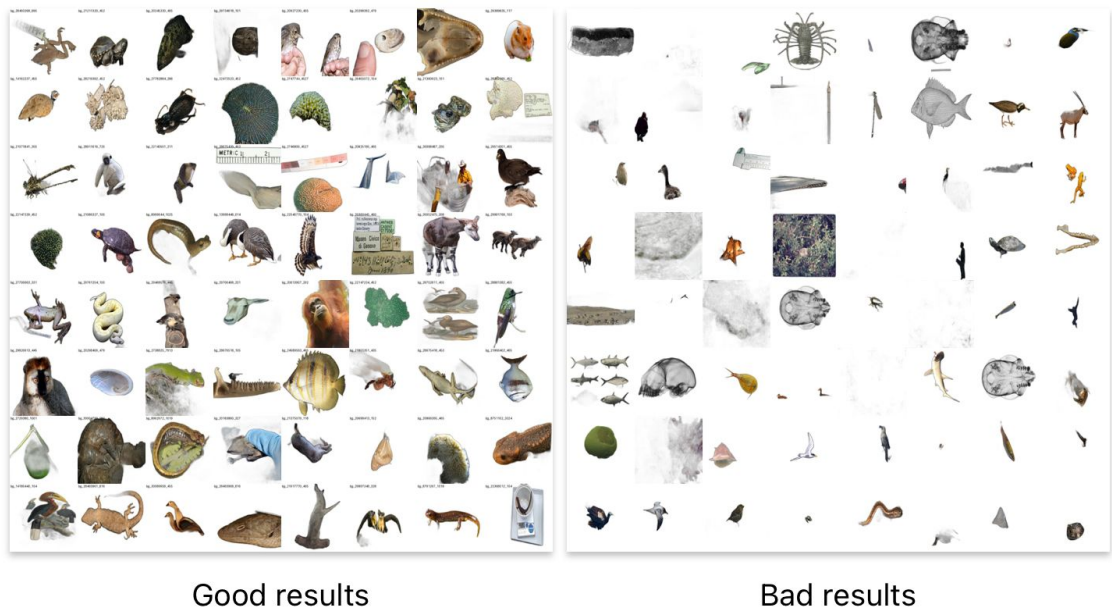


Figure 6: Augmentation Image Results

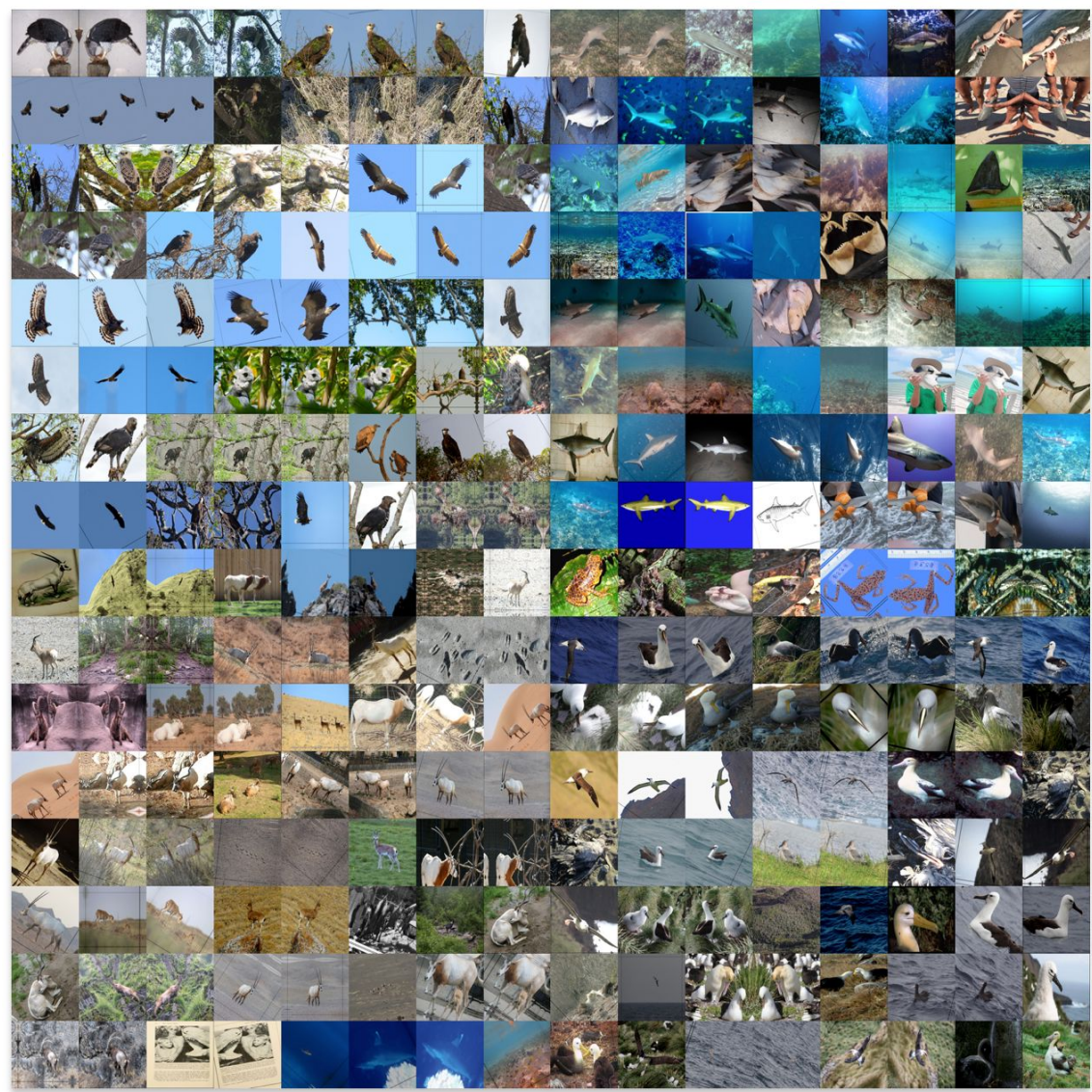


Figure 7: Augmentation Results

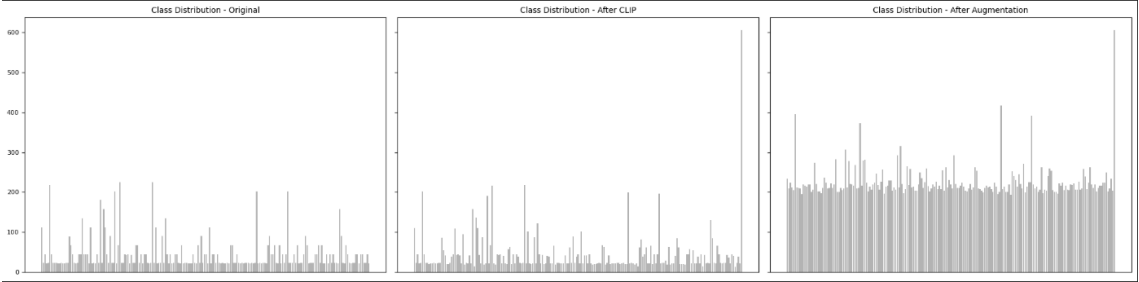


Figure 8: GAN results



Figure 9: Preprocessing Pipeline

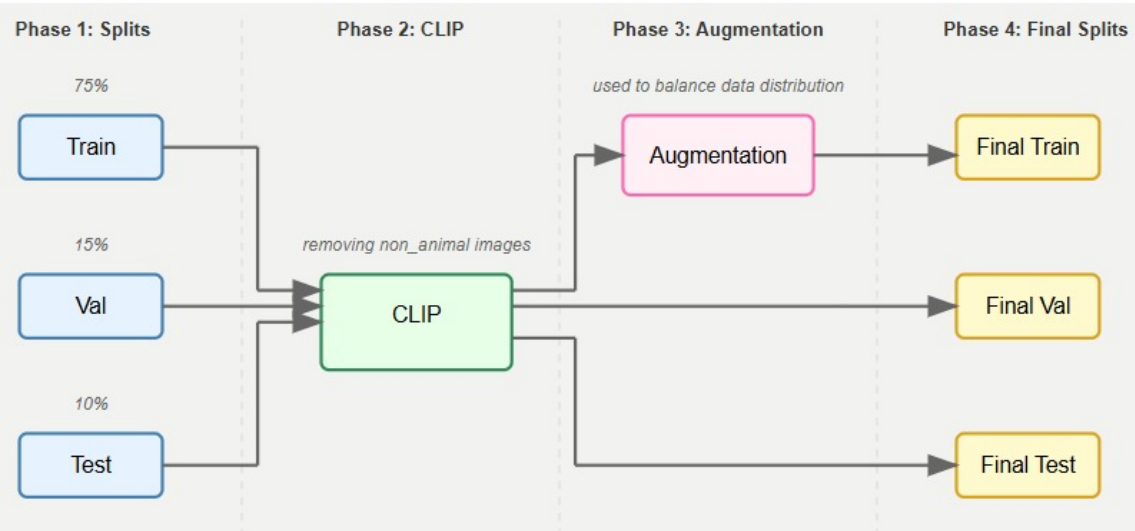


Figure 10: F1 score

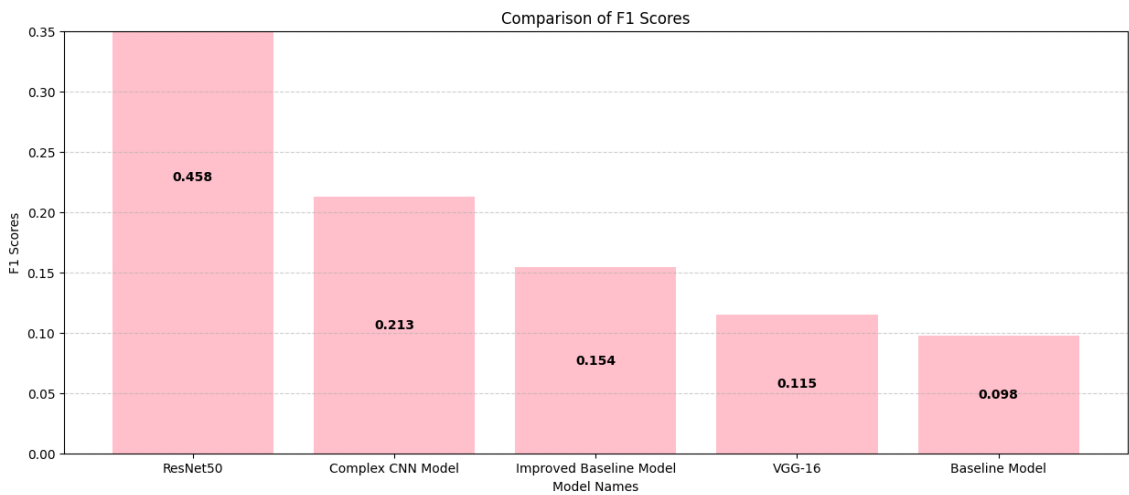


Figure 11: Accuracy score

